Plan de aseguramiento de calidad del software

Universidad Mariano Gálvez

Curso: Análisis de Sistemas II

Sistema: Biblioteca GG

Estándar: IEEE 730-2014

Integrantes: Giovanni Francia y Gerardo Lucero

Fecha: 22 de octubre de 2026

1. Propósito

Este plan describe cómo el equipo asegura que Biblioteca GG cumpla los requerimientos, el contrato de API y los criterios de hecho. Cubre la entrega final y las pruebas que viven en el repositorio.

2. Referencias

IEEE 730-2014, ISO/IEC/IEEE 29119-3, IEEE 1044-2009, ISO/IEC/IEEE 29148, OpenAPI 3.0.3, RFC 9457 y el backlog en docs/agile/backlog.md.

3. Gestión

El plan lo mantiene el equipo del proyecto. Quien ejerce el rol de Product Owner acepta la historia cuando se cumple la Definición de Hecho. No hay una organización de SQA separada: en un equipo de dos personas la independencia se busca con la revisión cruzada y con pruebas automatizadas que no dependen de recordar el resultado.

4. Diferencia entre QA y QC

QA es la prevención: requerimientos con identificador, contrato antes del cliente, criterios de preparado, escenarios Gherkin y revisión de diseño. QC es la detección: ejecutar las pruebas, mirar el código de estado y registrar el defecto. Este plan usa las dos. QA no sustituye la ejecución de las pruebas.

5. Enfoque shift-left

Los escenarios de aceptación se escribieron en features/ junto con el contrato OpenAPI, y las pruebas de tests/ se ejecutan con npm test antes de dar por cerrada la historia. El defecto se busca en la máquina del equipo, no en una demostración tardía. La pirámide pone más peso abajo.

6. Documentación exigida

Se mantienen SRS, fichas IEEE 1016, ADR, diagramas UML y C4, OpenAPI, colección Postman, escenarios Gherkin, este plan, los casos de prueba, el reporte de defectos y la matriz de trazabilidad.

7. Estándares, prácticas y convenciones

El código de producción está en src/. Los errores públicos siguen RFC 9457. Las pruebas usan el ejecutor node:test y supertest contra la aplicación sin abrir un puerto. La contraseña no se afirma

como presente en ninguna respuesta de usuario.

8. Revisiones

Se revisan el contrato y el requerimiento antes de ampliar un endpoint. Una historia no está hecha si el escenario Gherkin y la prueba automática no cubren el criterio de confirmación.

9. Pruebas

La pirámide de esta entrega:

- Unitarias: política de préstamo, circuit breaker y Cache-Aside con vencimiento.
- Integración: login, catálogo, alta de libro, préstamo, tope de política, caída de Neo4j y alta de usuario, contra la API real en memoria.
- Extremo a extremo manual: colección Postman y prototipo HTML con la API en el puerto 3000.

No hay una suite de navegador automatizada. El hueco está declarado para no presentar el prototipo como prueba de API.

10. Reporte de problemas

Los defectos se clasifican con IEEE 1044. La plantilla y un ejemplo están en el reporte de defectos. Un fallo de npm test es un defecto abierto hasta que la prueba vuelve a pasar.

11. Herramientas

Node.js, Express, el ejecutor de pruebas de Node, supertest, Swagger UI y Postman o Thunder Client. Los PDF de esta carpeta se generan desde las fuentes Markdown del repositorio para poder revisar el texto.

12. Control de medios y registros

El registro de calidad es el propio repositorio: pruebas, escenarios, contrato y este plan. No se versionan secretos. .env.example muestra las variables sin contraseñas reales. node_modules no se versiona.

13. Riesgos de calidad

Riesgo	Efecto	Mitigación
Demostrar sin PostgreSQL instalado	Se duda de la persistencia	Modo memoria con los mismos casos y script DDL para el motor real
Réplica eventual	El grafo puede ir detrás	meta.neo4j y actividad pendiente
Equipo pequeño	Poca independencia de la prueba	Pruebas automáticas y criterios de hecho escritos
Prototipo distinto de la API	Una pantalla promete de más	El prototipo solo cubre consulta y préstamo

14. Capacitación

Quien ejecute la entrega necesita Node.js 20 o superior y, para los PDF, las fuentes Arial del sistema. No necesita los cuatro motores para la demostración mínima.